

Netzwerkverbindung und MQTT

Welche Möglichkeiten der WLAN Verbindung hat der ESP32?

Der ESP32 kann sich auf verschiedene Arten mit WLAN-Netzwerken verbinden:

- **Station-Modus (STA):** Der ESP32 verbindet sich mit einem bestehenden WLAN-Netzwerk, ähnlich wie ein Laptop oder Smartphone.
- **Access Point-Modus (AP):** Der ESP32 erstellt ein eigenes WLAN-Netzwerk, in das sich andere Geräte verbinden können.
- **Mixed-Modus (AP + STA):** Eine Kombination aus beiden, bei der der ESP32 gleichzeitig mit einem Router verbunden ist und ein eigenes Netzwerk zur Verfügung stellt.

MQTT Netzwerkprotokoll

Erkläre kurz die Funktionsweise des MQTT Netzwerkprotokolls.

MQTT (Message Queuing Telemetry Transport) ist ein leichtgewichtiges, publish-subscribe-basiertes Protokoll, das sich gut für IoT-Anwendungen eignet. Es funktioniert folgendermaßen:

- Ein **Broker** verwaltet die Nachrichten und vermittelt zwischen den Clients.
- Clients können Nachrichten auf ein **Topic** veröffentlichen (**Publish**).
- Andere Clients können sich für dieses Topic registrieren (**Subscribe**), um die veröffentlichten Nachrichten zu empfangen.

Zum Testen der grundlegenden Funktionen von MQTT:

Im Rahmen des Unterrichts steht ein (virtueller) Rechner mit einer Linux-Installation auf dem MQTT läuft zur Verfügung. Verbinde dich über die Kommandozeile (Windows: Eingabeaufforderung) mit dem Rechner und teste die grundlegenden MQTT-Kommandozeilen – Befehle.

- Da ein Linux-Rechner mit MQTT zur Verfügung steht, kannst du die grundlegenden MQTT-Befehle testen. Nutze `mosquitto_pub` für das Senden von Nachrichten und `mosquitto_sub` für das Abonnieren von Nachrichten. Beispielbefehle:

```
mosquitto_pub -h <BROKER_IP> -t "test/topic" -m "Hallo, MQTT!"
```

```
mosquitto_sub -h <BROKER_IP> -t "test/topic"
```

WLAN-Verbindung

Wir haben die WLAN-Verbindung hergestellt und mit den Konsolenbefehlen aus dem Moodle-Kurs getestet.

Hier der Code:

```
// MeinESP32ClientName, welcher in MQTT zur Identifizierung genutzt wird, muss geändert werden

#include "WiFi.h"

// -----

#include <PubSubClient.h> // bzw. dreieckige Klammern

const char* MQTT_BROKER = "10.5.4.4";

WiFiClient espClient;
PubSubClient client(espClient);

// -----

const char *ssid = "FRITZ!Box 7490";
const char *password = "80073437732487114646"; //siehe FritzBox

void callback(char* topic, byte* message, unsigned int length) {
    Serial.print("Message arrived on topic: ");
    Serial.print(topic);
    Serial.print(". Message: ");
    String messageTemp;

    for (int i = 0; i < length; i++) {
        Serial.print((char)message[i]);
        messageTemp += (char)message[i];
    }
    Serial.println();
}

void reconnect() {
    // Loop until we're reconnected
    while (!client.connected()) {
        Serial.print("Attempting MQTT connection...");
        // Attempt to connect
        if (client.connect("TobiMagZüge")) {
```

```

    Serial.println("connected");

    // Subscribe
    client.subscribe("esp32/output");
} else {
    Serial.print("failed, rc=");
    Serial.print(client.state());
    Serial.println(" try again in 5 seconds");
    delay(5000);
}
}
}

void setup() {
    Serial.begin(9600);
    Serial.println("");
    Serial.println("-----");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.println("Connecting to WiFi..");
    }
    Serial.println("Connected to the WiFi network");

    // -----
    client.setServer(MQTT_BROKER, 1883);
    client.connect("TobiMagZüge");
    client.publish("TobiUndEyuep:", "Hallo Welt!");
    client.subscribe("esp32/output");
    client.setCallback(callback);
    // -----
}

void loop() {

```

```
if (!client.connected()) {  
    reconnect();  
}  
client.loop();  
}
```

MQTT

Hier die Ausgabe

```
Message arrived on topic: esp32/output. Message: Eyp  
Message arrived on topic: esp32/output. Message: lol
```